# Solución Bloque 1: Entendimiento Matemático y Formulación

Detección de Anomalías en Tráfico de Red mediante Modelos de Espacio de Estados

Estudiante

Abril 2026

# Índice


**1. Modelado en el Dominio Temporal** **4**
- 1.1. Discretización Zero-Order Hold (ZOH) . . . 4
  - 1.1.1. Derivación ZOH . . . 4
  - 1.1.2. Discretización Euler (Aproximación) . . . 4
- 1.2. Estabilidad con Matriz Diagonal . . . 5
  - 1.2.1. Caso Continuo . . . 5
  - 1.2.2. Caso Discreto . . . 5
- 1.3. Escalabilidad Computacional . . . 6
  - 1.3.1. Análisis de Complejidad . . . 6
  - 1.3.2. Comparación con RNN Tradicionales . . . 6

**2. Selección Dependiente de la Entrada** **8**
- 2.1. Ecuaciones Resultantes . . . 8
- 2.2. Sistema Modificado Paso a Paso . . . 8
  - 2.2.1. Paso 1: Proyectar entrada para obtener paso de tiempo . . . 8
  - 2.2.2. Paso 2: Computar matriz de transición con $\delta_t$ variable . . . 8
  - 2.2.3. Paso 3: Proyectar dinámicas de entrada y salida . . . 8
  - 2.2.4. Paso 4: Actualizar estado . . . 8
  - 2.2.5. Paso 5: Computar salida . . . 9
- 2.3. Beneficio: Selección Adaptativa de Información . . . 9
  - 2.3.1. 1. Olvido Selectivo mediante $\delta_t$ . . . 9
  - 2.3.2. 2. Proyección Selectiva de Entrada mediante $B_t$ . . . 9
  - 2.3.3. 3. Extracción Selectiva del Estado mediante $C_t$ . . . 10
  - 2.3.4. Ejemplo Concreto: Detección de Anomalías en Tráfico . . . 10

**3. Dominio Frecuencial** **11**
- 3.1. Transformada de Fourier Continua . . . 11
  - 3.1.1. Definición . . . 11
- 3.2. Transformada de Fourier Discreta (DFT) . . . 11
  - 3.2.1. Definición . . . 11
- 3.3. Propiedad de Simetría Conjugada . . . 12

3.3.1. Enunciado . . . 12
3.3.2. Demostración . . . 12
3.3.3. Consecuencias . . . 12

**4. Mecanismo de Extracción de Features Espectrales 14**
4.1. Algoritmo Paso a Paso . . . 14
4.1.1. Paso (a): Calcular DFT a lo largo del eje temporal . . . 14
4.1.2. Paso (b): Retener solo las $K$ componentes de mayor magnitud . . . 14
4.1.3. Paso (c): Aplicar filtro complejo aprendible por canal . . . 14
4.1.4. Paso (d): Transformada inversa a dominio temporal . . . 15
4.2. Ecuaciones Resumidas . . . 15

**5. Fusión Ponderada 16**
5.1. Formulación . . . 16
5.2. Justificación 1: ¿Por qué producto elemento a elemento y no un escalar? . . . 16
5.3. Justificación 2: Rol de la inicialización . . . 17
5.3.1. Inicialización Uniforme . . . 17
5.3.2. Inicialización Sesgada . . . 17
5.3.3. Inicialización con Softmax Normalizado . . . 18
5.4. Justificación 3: ¿Qué pasa si se fijan en $\alpha = \beta = 1$? . . . 18
5.4.1. Consecuencias Negativas . . . 18

**6. Arquitectura Completa 20**
6.1. Diagrama de Arquitectura . . . 20
6.2. Componentes Principales . . . 21
6.2.1. Entrada . . . 21
6.2.2. Proyección de Entrada . . . 21
6.2.3. Rama Temporal . . . 21
6.2.4. Rama Frecuencial . . . 21
6.2.5. Fusión . . . 21
6.2.6. Cabeza de Clasificación . . . 21
6.3. ¿Por qué la rama frecuencial está en paralelo y no en serie? . . . 22
6.3.1. Configuración en Serie (NO recomendado) . . . 22
6.3.2. Configuración en Paralelo (RECOMENDADO) . . . 22

**7. Hiperparámetros 24**
7.1. Listado y Clasificación . . . 24
7.2. Justificación de Valores Iniciales . . . 24
7.2.1. Parámetros de Capacidad . . . 24
7.2.2. Parámetros de Estabilidad . . . 25
7.2.3. Parámetros de Entrenamiento . . . 26
7.2.4. Parámetros de Regularización . . . 27
7.3. Inicialización de Parámetros Clave . . . 27
7.3.1. Eigenvalues $\lambda_i$ . . . 27
7.3.2. Pesos de Fusión $\alpha, \beta$ . . . 27

**8. Filtro de Kalman como Post-Procesamiento** **28**
8.1. Motivación . . . 28
8.2. Modelo Estocástico Escalar . . . 28
8.2.1. Ecuación de Estado (Dinámica del Sistema) . . . 28
8.2.2. Ecuación de Observación (Medición Ruidosa) . . . 29
8.2.3. Supuestos . . . 29
8.3. Filtro de Kalman: Derivación Completa . . . 29
8.3.1. Notación . . . 29
8.3.2. Paso 1: Predicción (A Priori) . . . 29
8.3.3. Paso 2: Ganancia de Kalman (Óptima) . . . 30
8.3.4. Paso 3: Actualización (A Posteriori) . . . 31
8.4. Resumen del Algoritmo . . . 32
8.5. Análisis de Casos Extremos . . . 32
8.5.1. Pregunta 1: ¿Qué pasa cuando $\sigma_w^2 \to 0$? . . . 32
8.5.2. Pregunta 2: ¿Qué pasa cuando $\sigma_w^2 \to \infty$? . . . 33
8.5.3. Pregunta 3: ¿Qué relación $\sigma_w^2/\sigma_v^2$ favorece cada régimen? . . . 34
8.5.4. Pregunta 4: ¿En qué escenarios del problema real esperarían que cada régimen sea preferible? . . . 34

# 1. Modelado en el Dominio Temporal

## 1.1. Discretización Zero-Order Hold (ZOH)

Partimos del sistema dinámico lineal en tiempo continuo:

$$x'(t) = A_c x(t) + B_c u(t) \tag{1}$$
$$y(t) = Cx(t) + Du(t) \tag{2}$$

donde $x(t) \in \mathbb{R}^N$ es el estado, $u(t) \in \mathbb{R}^D$ es la entrada, $y(t) \in \mathbb{R}^P$ es la salida, y $A_c \in \mathbb{R}^{N\times N}$, $B_c \in \mathbb{R}^{N\times D}$, $C \in \mathbb{R}^{P\times N}$, $D \in \mathbb{R}^{P\times D}$ son matrices de parámetros.

### 1.1.1. Derivación ZOH

Asumimos que la entrada es constante a tramos: $u(t) = u_k$ para $t \in [kT, (k+1)T]$, donde $T > 0$ es el período de muestreo.

Resolvemos la ecuación diferencial homogénea:

$$\frac{dx}{dt} = A_c x(t) \tag{3}$$

con condición inicial $x(kT) = x_k$. La solución es:

$$x(t) = e^{A_c(t-kT)} x_k \tag{4}$$

Evaluando en $t = (k+1)T$:

$$x((k+1)T) = e^{A_c T} x_k + \int_0^T e^{A_c \tau} B_c u_k \, d\tau \tag{5}$$

Definimos:

$$A = e^{A_c T} \tag{6}$$
$$B = \left( \int_0^T e^{A_c \tau} d\tau \right) B_c \tag{7}$$

Si $A_c$ es invertible, esta integral se puede expresar como:

$$B = A_c^{-1}(A - I)B_c \tag{8}$$

**Forma Discreta ZOH:**

$$x_{k+1} = Ax_k + Bu_k \tag{9}$$
$$y_k = Cx_k + Du_k \tag{10}$$

### 1.1.2. Discretización Euler (Aproximación)

Una aproximación más simple usa la aproximación de Euler forward:

$$x_{k+1} \approx x_k + T \cdot A_c x_k + T \cdot B_c u_k = (I + TA_c)x_k + TB_c u_k \tag{11}$$

**Forma Discreta Euler:**

$$A_{\text{Euler}} = I + TA_c \tag{12}$$
$$B_{\text{Euler}} = TB_c \tag{13}$$

**Comparación ZOH vs Euler:**

- **ZOH** es exacta para sistemas lineales con entrada constante a tramos
- **Euler** es una aproximación de primer orden; ambas convergen para $T$ pequeño
- ZOH preserva mejor la dinámica para intervalos de muestreo más grandes
- La diferencia es importante cuando $T$ es relativamente grande o $\|A_cT\|$ no es pequeño

## 1.2. Estabilidad con Matriz Diagonal

### 1.2.1. Caso Continuo

Consideramos la parametrización:

$$A_c = -\text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_N), \quad \lambda_i > 0 \tag{14}$$

Para el sistema $x'(t) = A_c x(t)$, la solución es:

$$x_i(t) = x_i(0)e^{-\lambda_i t} \tag{15}$$

Cada componente decae exponencialmente porque $\lambda_i > 0$. El sistema es **asintóticamente estable** ya que todos los valores propios satisfacen $\text{Re}(\lambda_i) = -\lambda_i < 0$ (semiplano izquierdo del plano complejo).

### 1.2.2. Caso Discreto

Bajo discretización ZOH:

$$A_d = e^{A_c T} = e^{-\text{diag}(\lambda_1,\ldots,\lambda_N)T} = \text{diag}(e^{-\lambda_1 T}, \ldots, e^{-\lambda_N T}) \tag{16}$$

Los valores propios de $A_d$ son:

$$\mu_i = e^{-\lambda_i T} \tag{17}$$

**Condición de Estabilidad Discreta:** Para estabilidad, necesitamos $|\mu_i| < 1$ para todo $i$.

Como $\lambda_i > 0$ y $T > 0$:

$$|\mu_i| = |e^{-\lambda_i T}| = e^{-\lambda_i T} < 1 \quad \forall i \tag{18}$$

Esta desigualdad se cumple siempre porque $e^x < 1 \Leftrightarrow x < 0$, y $-\lambda_i T < 0$.

**Garantía de Estabilidad:**
La parametrización diagonal $A = -\text{diag}(\lambda_i)$ con $\lambda_i > 0$ **garantiza automáticamente** la estabilidad del sistema discreto porque:

1. Los valores propios discretos quedan dentro del círculo unitario
2. Cada componente de estado decae: $x_i(k) = x_i(0)e^{-\lambda_i kT} \to 0$
3. No hay riesgo de divergencia ni explosión de gradientes

**Pregunta: ¿Qué condición debe cumplir el espectro?** *Respuesta:* El espectro (conjunto de valores propios) de la matriz discretizada $A_d$ debe satisfacer:

$$\sigma(A_d) = \{\mu_1, \mu_2, \ldots, \mu_N\} \subset \{\mu \in \mathbb{C} : |\mu| < 1\} \tag{19}$$

Es decir, **todos los valores propios deben estar dentro del círculo unitario** en el plano complejo. Con la parametrización diagonal propuesta, esta condición se garantiza automáticamente sin necesidad de restricciones adicionales.

## 1.3. Escalabilidad Computacional

### 1.3.1. Análisis de Complejidad

Para la recurrencia:

$$x_{k+1} = Ax_k + Bu_k \tag{20}$$

con dimensiones: $x_k \in \mathbb{R}^N$, $u_k \in \mathbb{R}^D$, $A \in \mathbb{R}^{N\times N}$, $B \in \mathbb{R}^{N\times D}$.

1. Multiplicación $Ax_k$: $\mathcal{O}(N^2)$ operaciones
2. Multiplicación $Bu_k$: $\mathcal{O}(N \cdot D)$ operaciones
3. Una recurrencia para $k = 1, 2, \ldots, L$: $\mathcal{O}(L \cdot N^2)$ operaciones
4. **Total:** $\mathcal{O}(L \cdot N^2)$ operaciones para una secuencia de longitud $L$

Fijando $N$ (dimensión del estado latente, típicamente $N \sim 64$ a 512), el costo es **lineal en** $L$:

$$\text{Costo} \propto L \quad (\text{con } N \text{ fijo}) \tag{21}$$

Esto contrasta favorablemente con Transformers estándar que tienen complejidad $\mathcal{O}(L^2)$.

### 1.3.2. Comparación con RNN Tradicionales

**LSTM y GRU:** Ambos tienen complejidad $\mathcal{O}(L \cdot N^2)$, comparable a nuestro SSM. Sin embargo, sufren del **problema del gradiente**.

**Problema del Gradiente en LSTM/GRU:** Durante Backpropagation Through Time (BPTT), el gradiente se propaga multiplicándose por la derivada en cada paso temporal:

$$\frac{\partial L}{\partial x_0} = \frac{\partial L}{\partial x_L} \prod_{k=1}^{L} \frac{\partial x_k}{\partial x_{k-1}} \tag{22}$$

Para RNN estándar (tanh/sigmoide):

$$\frac{\partial x_k}{\partial x_{k-1}} = W^T \text{diag}(f'(h_{k-1})) \tag{23}$$

Donde $f$ es la función de activación (por ejemplo, tanh o sigmoide). Las derivadas de estas funciones están acotadas (e.g., $f'(\cdot) \in [0, 0{,}25]$ para sigmoide):

- **Vanishing Gradient:** El producto de factores $< 1$ se hace exponencialmente pequeño: $\prod_{k=1}^{L} 0{,}25 \approx 0$ exponencialmente rápido
- Imposibilita aprender dependencias a largo plazo (más allá de $\sim 10$ pasos)
- Las capas tempranas no reciben gradientes significativos

LSTM/GRU mitigan esto con compuertas, pero no resuelven completamente el problema y añaden complejidad.

**Ventaja del SSM Diagonal:** Con $A = -\text{diag}(\lambda_i)$ y discretización ZOH, la derivada es:

$$\frac{\partial x_{k+1}}{\partial x_k} = A_d = \text{diag}(e^{-\lambda_i T}) \tag{24}$$

El gradiente se propaga como:

$$\frac{\partial L}{\partial x_0} = \frac{\partial L}{\partial x_L} \prod_{k=1}^{L} e^{-\lambda_i T} = \frac{\partial L}{\partial x_L} e^{-\lambda_i L T} \tag{25}$$

Ventajas:

1. Cada factor $e^{-\lambda_i T} \in (0, 1)$ es controlado y predecible
2. El decaimiento es **exponencial pero estable**
3. Podemos ajustar $\lambda_i$ y $T$ para controlar el rango de dependencias que el modelo captura
4. No hay explosión ($\mu_i > 1$) ni colapso catastrófico ($\mu_i \approx 0$)
5. Mejor capacidad de retención de información a largo plazo

**Conclusión:**
El SSM diagonal evita el problema del gradiente de RNNs tradicionales mediante una parametrización que garantiza control automático sobre cómo decae la información a través del tiempo. Esto permite capturar dependencias a mucho mayor plazo sin explosión ni desvanecimiento.

# 2. Selección Dependiente de la Entrada

En la formulación clásica de espacio de estados, los parámetros $A$, $B$, $C$, $\Delta t$ son **fijos**. Proponemos una variante en la que estos parámetros se generan como **proyecciones lineales de la entrada**.

## 2.1. Ecuaciones Resultantes

Definimos funciones de proyección lineal:

$$\delta_t = f_\Delta(u_t) = \text{softplus}(W_\Delta u_t + b_\Delta) \tag{26}$$
$$B_t = f_B(u_t) = W_B u_t + b_B \tag{27}$$
$$C_t = f_C(u_t) = W_C u_t + b_C \tag{28}$$

donde:

- $W_\Delta \in \mathbb{R}^{1\times D}$ (escalar de tiempo, por eso usamos softplus para asegurar $\delta_t > 0$)
- $W_B \in \mathbb{R}^{N\times D}$, $b_B \in \mathbb{R}^N$ (parámetros de entrada al estado)
- $W_C \in \mathbb{R}^{D\times N}$, $b_C \in \mathbb{R}^D$ (parámetros de proyección del estado a salida)
- $\text{softplus}(x) = \log(1 + e^x)$ asegura $\delta_t > 0$ (necesario para estabilidad)

## 2.2. Sistema Modificado Paso a Paso

### 2.2.1. Paso 1: Proyectar entrada para obtener paso de tiempo

$$\delta_t = \text{softplus}(W_\Delta u_t + b_\Delta) \tag{29}$$

### 2.2.2. Paso 2: Computar matriz de transición con $\delta_t$ variable

$$A_t = e^{-\delta_t \text{diag}(\lambda_1,\lambda_2,\ldots,\lambda_N)} = \text{diag}(e^{-\delta_t\lambda_1}, \ldots, e^{-\delta_t\lambda_N}) \tag{30}$$

Nota: $A_t$ se recomputa en cada paso basado en $\delta_t(u_t)$.

### 2.2.3. Paso 3: Proyectar dinámicas de entrada y salida

$$B_t = W_B u_t + b_B \quad \in \mathbb{R}^N \tag{31}$$
$$C_t = W_C u_t + b_C \quad \in \mathbb{R}^D \tag{32}$$

### 2.2.4. Paso 4: Actualizar estado

$$x_t = A_t x_{t-1} + B_t \odot u_t \tag{33}$$

donde $\odot$ denota multiplicación elemento a elemento.

#### 2.2.5. Paso 5: Computar salida

$$y_t = C_t \odot x_t + Du_t \tag{34}$$

**Resumen del Sistema Modificado:**

$$\delta_t = \text{softplus}(W_\Delta u_t + b_\Delta) \tag{35}$$
$$A_t = e^{-\delta_t \text{diag}(\lambda)} \tag{36}$$
$$B_t = W_B u_t + b_B \tag{37}$$
$$C_t = W_C u_t + b_C \tag{38}$$
$$x_t = A_t x_{t-1} + B_t \odot u_t \tag{39}$$
$$y_t = C_t \odot x_t + Du_t \tag{40}$$

## 2.3. Beneficio: Selección Adaptativa de Información

**Pregunta: ¿Qué gana el modelo con esta modificación?** *Respuesta:*
El modelo adquiere **capacidad de atención temporal selectiva** en tres niveles:

#### 2.3.1. 1. Olvido Selectivo mediante $\delta_t$

Mediante la proyección $\delta_t(u_t)$, el modelo puede **adaptativamente aumentar o disminuir la tasa de olvido** dependiendo del contenido de la entrada:

- Si la entrada señala un **cambio de régimen** anómalo: el modelo aumenta $\delta_t$, lo que hace que los valores propios $\mu_i = e^{-\delta_t \lambda_i}$ se reduzcan, olvidando más rápido el pasado irrelevante
- Si la entrada es **consistente** con la historia reciente: el modelo puede disminuir $\delta_t$, reteniendo información histórica más tiempo
- Interpretación: $\delta_t$ actúa como un **"olvido inteligente"** que depende del contexto

#### 2.3.2. 2. Proyección Selectiva de Entrada mediante $B_t$

Mediante $B_t = W_B u_t + b_B$, el modelo puede **amplificar u omitir componentes específicos** de la entrada:

- Si un canal de entrada es ruidoso o irrelevante, puede hacer que $(B_t)_i \approx 0$ para ese canal
- Si un canal es informativo para actualizar el estado, puede hacer que $(B_t)_i$ sea grande
- Interpretación: **proyección selectiva de qué información de entrada afecta el estado interno**

#### 2.3.3. 3. Extracción Selectiva del Estado mediante $C_t$

Mediante $C_t = W_C u_t + b_C$, el modelo puede **enfatizar componentes específicos del estado** para la salida:

- Si ciertos componentes de estado son informativos según la entrada actual, $C_t$ los amplifica
- Si otros componentes son irrelevantes, $C_t$ los atenúa
- Interpretación: **readout selectivo del estado según el contexto**

#### 2.3.4. Ejemplo Concreto: Detección de Anomalías en Tráfico

Considera un flujo de tráfico con dos regímenes: normal y anómalo.

1. **Tráfico Normal (patrón esperado):**
   - Entrada $u_t$ activa proyección que mantiene $\delta_t$ pequeño
   - El modelo retiene todo el estado histórico con $\mu_i \approx 1$ (olvido lento)
   - Permite detectar cambios muy graduales
2. **Tráfico Anómalo Detectado:**
   - Entrada $u_t$ activa proyección que aumenta $\delta_t$ significativamente
   - Los valores propios se reducen: $\mu_i = e^{-\delta_t \lambda_i}$ pequeños
   - El modelo "olvida" rápidamente el pasado normal y reacciona rápidamente al nuevo estado
   - El estado se reiniicia efectivamente para capturar la dinámca del ataque
3. **Selección de Características:**
   - $B_t$ puede enfatizar características de volumen/protocolo durante ataques
   - $C_t$ puede extraer principalmente información de latencia durante tráfico normal
   - El modelo adapta continuamente qué características son relevantes

**Conclusión:**
La selección dependiente de la entrada otorga al modelo **tres mecanismos de atención adaptativa**:

1. Olvido temporal inteligente (según volatilidad detectada)
2. Selección de qué información de entrada procesar
3. Selección de qué información del estado extraer

Sin estos, el modelo sería un SSM rígido incapaz de adaptar su comportamiento al contenido de la entrada.

# 3. Dominio Frecuencial

## 3.1. Transformada de Fourier Continua

### 3.1.1. Definición

La transformada de Fourier continua de una señal $x(t)$ es:

$$X(f) = \int_{-\infty}^{+\infty} x(t)e^{-j2\pi ft}\, dt \tag{41}$$

donde:

- $j = \sqrt{-1}$ es la unidad imaginaria
- $f$ es la frecuencia en Hz
- $X(f) \in \mathbb{C}$ es la transformada, un número complejo
- $x(t) \in \mathbb{R}$ es la señal original (típicamente real)

**Interpretación:** Descompone $x(t)$ como una suma (integral) de exponenciales complejas $e^{j2\pi ft}$ con diferentes frecuencias $f$ y amplitudes complejas $X(f)$.

## 3.2. Transformada de Fourier Discreta (DFT)

### 3.2.1. Definición

Para una señal discreta $x[n]$ con $n = 0, 1, \ldots, N-1$ (total $N$ muestras), la DFT es:

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N}, \quad k = 0, 1, \ldots, N-1 \tag{42}$$

donde:

- $X[k] \in \mathbb{C}$ es el coeficiente de Fourier en la frecuencia discreta $k$
- Hay exactamente $N$ coeficientes (uno para cada frecuencia)
- La frecuencia física correspondiente a $k$ es $f_k = k/(NT_s)$, donde $T_s$ es el período de muestreo

**Nota sobre invertibilidad:** La transformada inversa es:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]e^{j2\pi kn/N} \tag{43}$$

Esto permite recuperar la señal original a partir de sus coeficientes de Fourier.

## 3.3. Propiedad de Simetría Conjugada

### 3.3.1. Enunciado

Para una señal **real** $x[n] \in \mathbb{R}$ (entrada real), los coeficientes de Fourier satisfacen:

$$X[N-k] = X^*[k] \tag{44}$$

donde $X^*[k]$ denota el **complejo conjugado** de $X[k]$.

### 3.3.2. Demostración

Computamos $X[N-k]$:

$$\begin{aligned}
X[N-k] &= \sum_{n=0}^{N-1} x[n] e^{-j2\pi(N-k)n/N} && (45)\\
&= \sum_{n=0}^{N-1} x[n] e^{-j2\pi n} e^{j2\pi kn/N} \quad \text{(separar exponencial)} && (46)\\
&= \sum_{n=0}^{N-1} x[n] e^{j2\pi kn/N} \quad \text{(usar } e^{-j2\pi n} = 1\text{)} && (47)\\
&= \left( \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N} \right)^* \quad \text{(tomar conjugado)} && (48)\\
&= X^*[k] && (49)
\end{aligned}$$

La igualdad $\sum x[n]e^{j\alpha n} = (\sum x[n]e^{-j\alpha n})^*$ es válida porque $x[n] \in \mathbb{R}$, así que $(x[n])^* = x[n]$.

### 3.3.3. Consecuencias

La propiedad de simetría conjugada implica que muchos coeficientes son redundantes:

- $X[0]$ es real (es la suma de todos los $x[n]$)
- $X[k]$ para $k = 1, 2, \ldots, \lfloor N/2 \rfloor$ contiene información única
- $X[N/2]$ (si $N$ es par) es real (es $\sum(-1)^n x[n]$)
- $X[N-k]$ para $k = 1, 2, \ldots, \lfloor N/2 \rfloor$ son complejos conjugados de $X[k]$, así que son redundantes

**Pregunta: ¿Por qué basta calcular $L/2+1$ coeficientes?** *Respuesta:*
Para una señal de longitud $L$:

- Índices únicos e informativos: $k = 0, 1, 2, \ldots, \lfloor L/2 \rfloor$
- Total de índices: $\lfloor L/2 \rfloor + 1 = L/2 + 1$ (si $L$ es par)
- Los índices $k = \lfloor L/2 \rfloor + 1, \ldots, L-1$ son complejos conjugados de índices menores, por tanto redundantes

Esto reduce la memoria requerida de $L$ coeficientes complejos a $L/2 + 1$, ahorrando aproximadamente **50 % de almacenamiento y computación**.

**Transformada de Fourier Discreta Real (rfft):**
PyTorch proporciona `torch.fft.rfft` que calcula solo los $L/2 + 1$ coeficientes únicos, evitando redundancia. La transformada inversa `torch.fft.irfft` reconstruye la señal real original de estos coeficientes.

# 4. Mecanismo de Extracción de Features Espectrales

Proponemos un mecanismo para extraer características periódicas de la señal en el dominio de frecuencia:

## 4.1. Algoritmo Paso a Paso

Sea la entrada un buffer temporal de longitud $L$ con $D$ canales: $x \in \mathbb{R}^{L \times D}$.

### 4.1.1. Paso (a): Calcular DFT a lo largo del eje temporal

Para cada canal $d = 1, 2, \ldots, D$, computamos la transformada de Fourier discreta:

$$X_d[k] = \sum_{n=0}^{L-1} x[n, d]\, e^{-j2\pi kn/L}, \quad k = 0, 1, \ldots, \lfloor L/2 \rfloor \tag{50}$$

Usando `torch.fft.rfft`:

$$X = \mathrm{rfft}(x, \mathrm{dim} = 0) \quad \in \mathbb{C}^{(L/2+1) \times D} \tag{51}$$

**Salida:** Matriz de coeficientes complejos con forma $(L/2 + 1, D)$.

### 4.1.2. Paso (b): Retener solo las $K$ componentes de mayor magnitud

Para cada canal $d$:

1. Computar la magnitud de cada coeficiente:

$$|X_d[k]| = \sqrt{(\mathrm{Re}(X_d[k]))^2 + (\mathrm{Im}(X_d[k]))^2} \tag{52}$$

2. Identificar los índices de las $K$ frecuencias con mayor magnitud:

$$I_d = \mathrm{topk}(|X_d|, K) \quad \text{(índices de los } K \text{ máximos)} \tag{53}$$

3. Crear máscara y aplicarla:

$$X'_d[k] = \begin{cases} X_d[k] & \text{si } k \in I_d \\ 0 & \text{en otro caso} \end{cases} \tag{54}$$

**Salida:** Matriz $X' \in \mathbb{C}^{(L/2+1) \times D}$ con solo $K$ componentes no nulos por canal (el resto son ceros).

**Interpretación:** Nos quedamos con las $K$ frecuencias más importantes (de mayor amplitud) para cada canal, descartando ruido de alta frecuencia.

### 4.1.3. Paso (c): Aplicar filtro complejo aprendible por canal

Definimos parámetros aprendibles (entrenables):

$$H_d = a_d + j\, b_d \in \mathbb{C}, \quad d = 1, 2, \ldots, D \tag{55}$$

donde $a_d, b_d \in \mathbb{R}$ son parámetros reales aprendibles durante el entrenamiento.
Multiplicamos elemento a elemento en el dominio de frecuencia:

$$X''_d[k] = H_d \cdot X'_d[k], \quad \forall k, d \tag{56}$$

**Interpretación:** Cada canal tiene su propio **filtro complejo** que puede amplificar o atenuar diferentes bandas de frecuencia. El filtro es aprendible, así que el modelo descubre automáticamente qué ponderaciones espectrales son óptimas.

```
# H: parametros complejos aprendibles, shape (D, )
# X_masked: salida del paso (b), shape (L//2+1, D)
X_filtered = X_masked * H.unsqueeze(0)  # broadcast
```

### 4.1.4. Paso (d): Transformada inversa a dominio temporal

Aplicamos la transformada de Fourier discreta inversa:

$$\tilde{x}_d[n] = \text{IFFT}(X''_d) = \frac{1}{L}\sum_{k=0}^{\lfloor L/2 \rfloor} X''_d[k]e^{j2\pi kn/L}, \quad n = 0, 1, \ldots, L-1 \tag{57}$$

En PyTorch:

```
x_spectral = torch.fft.irfft(X_filtered, n=L, dim=0)
# x_spectral: shape (L, D), dtype=float32 (real)
```

**Salida:** Matriz $\tilde{x} \in \mathbb{R}^{L\times D}$ en el dominio temporal nuevamente, ahora filtrada para enfatizar componentes periódicos.

## 4.2. Ecuaciones Resumidas

**Extractor Espectral Completo:**
Entrada: $x \in \mathbb{R}^{L\times D}$

$$X = \text{rfft}(x, \text{dim} = 0) \quad \in \mathbb{C}^{(L/2+1)\times D} \tag{58}$$

$$X' = \text{Mask_TopK}(X, K) \quad \text{(retener solo top-}K\text{)} \tag{59}$$

$$X'' = H \odot X' \quad \text{(multiplicar por filtro complejo aprendible)} \tag{60}$$

$$\tilde{x} = \text{irfft}(X'', n = L) \quad \in \mathbb{R}^{L\times D} \tag{61}$$

Salida: $\tilde{x} \in \mathbb{R}^{L\times D}$ (características espectrales extraídas)

# 5. Fusión Ponderada

## 5.1. Formulación

Proponemos fusionar las representaciones temporal y frecuencial mediante:

$$z_t = \alpha \odot x_t + \beta \odot \tilde{x}(f)_t \tag{62}$$

donde:

- $x_t \in \mathbb{R}^D$ es la representación **temporal** en el paso $t$ (salida del SSM)
- $\tilde{x}(f)_t \in \mathbb{R}^D$ es la representación **frecuencial** en el paso $t$ (salida del extractor espectral)
- $\alpha, \beta \in \mathbb{R}^D$ son vectores de pesos **aprendibles** (parámetros del modelo)
- $\odot$ denota el producto **elemento a elemento** (Hadamard)
- $z_t \in \mathbb{R}^D$ es la representación **fusionada**

## 5.2. Justificación 1: ¿Por qué producto elemento a elemento y no un escalar?

**Alternativa 1: Escalar (no recomendado)** Si usáramos un escalar $\alpha \in \mathbb{R}$:

$$z_t = \alpha \cdot x_t + (1 - \alpha) \cdot \tilde{x}(f)_t \tag{63}$$

Problemas:

1. **Falta de selectividad:** Todas las dimensiones latentes se ponderan de manera idéntica
2. **Rigidez:** No permite que diferentes características se combinen de manera independiente
3. **Pérdida de expresividad:** Estamos forzando a que el modelo tome una única decisión global (50-50, 70-30, etc.)

**Alternativa 2: Vector elemento a elemento (RECOMENDADO)** Con $\alpha \in \mathbb{R}^D$, cada componente latente $i$ puede tener su propio balance:

$$z_t[i] = \alpha_i \cdot x_t[i] + \beta_i \cdot \tilde{x}(f)_t[i] \tag{64}$$

Ventajas:

1. **Selectividad dimensional:** La dimensión latente $i$ puede especializarse en la rama que es más informativa para ella
2. **Ejemplo concreto:**
   - Dimensión 1 podría ser sensible a cambios temporales rápidos: $\alpha_1 = 0{,}9, \beta_1 = 0{,}1$ (casi toda temporal)

- Dimensión 2 podría capturar ciclos diarios: $\alpha_2 = 0{,}2, \beta_2 = 0{,}8$ (casi toda frecuencial)
- Dimensión 3 podría ser intermedia: $\alpha_3 = 0{,}5, \beta_3 = 0{,}5$ (balance)

3. **Espacio de parámetros más rico:** $2D$ parámetros vs. 2 escalares
4. **Mayor capacidad modeladora:** El modelo tiene más grados de libertad para adaptar la combinación

**Conclusión:** El producto elemento a elemento permite que cada dimensión latente aprenda su propio balance óptimo entre información temporal y frecuencial, resultando en un modelo más expresivo y flexible.

## 5.3. Justificación 2: Rol de la inicialización

La inicialización de $\alpha$ y $\beta$ determina el **comportamiento inducido** del modelo en las primeras épocas de entrenamiento:

### 5.3.1. Inicialización Uniforme

Inicializar con $\alpha_i = 0{,}5, \beta_i = 0{,}5$ para todo $i$:

$$z_t = 0{,}5 \cdot x_t + 0{,}5 \cdot \tilde{x}(f)_t \tag{65}$$

Características:

- El modelo comienza con **balance perfecto** entre ramas temporal y frecuencial
- Durante el entrenamiento, los pesos se ajustan libremente según el gradiente
- No introduce sesgo a priori; ambas ramas tienen igual "peso" inicial
- Ventaja: exploración imparcial del espacio de soluciones

### 5.3.2. Inicialización Sesgada

Inicializar con $\alpha_i = 0{,}9, \beta_i = 0{,}1$:

$$z_t = 0{,}9 \cdot x_t + 0{,}1 \cdot \tilde{x}(f)_t \tag{66}$$

Características:

- El modelo comienza **enfatizando la rama temporal**
- Reduce el espacio de búsqueda en las primeras épocas
- Puede acelerar convergencia si la rama temporal es realmente más importante
- Riesgo: sesga el aprendizaje; la rama frecuencial podría no desarrollarse bien incluso si es útil

### 5.3.3. Inicialización con Softmax Normalizado

Para garantizar que $\sum_i(\alpha_i + \beta_i)$ esté acotado, podemos usar parámetrizacion log:

$$\alpha_i = \frac{e^{\log \alpha_i}}{\sum_j(e^{\log \alpha_j} + e^{\log \beta_j})}, \quad \beta_i = \frac{e^{\log \beta_i}}{\sum_j(e^{\log \alpha_j} + e^{\log \beta_j})} \tag{67}$$

Características:

- Garantiza normalización: $\sum_i(\alpha_i + \beta_i) \approx 1$ (suma a constante)
- Evita que los pesos exploten sin límite
- Estabiliza el entrenamiento
- Requiere parámetrizacion indirecta (log-weights), más compleja

**Recomendación:** Usar inicialización **uniforme** ($\alpha = \beta = 0{,}5$) por defecto. Es simple, insesgada y funciona bien en la práctica.

## 5.4. Justificación 3: ¿Qué pasa si se fijan en $\alpha = \beta = 1$?

Si los pesos no son **aprendibles** (se fijan) a $\alpha_i = 1, \beta_i = 1$ para todo $i$:

$$z_t = x_t + \tilde{x}(f)_t \quad \text{(suma directa, sin pesos)} \tag{68}$$

### 5.4.1. Consecuencias Negativas

**1. Pérdida de Adaptabilidad** El modelo no puede aprender a desenfatizar una rama. Está forzado a sumar con pesos iguales, sin flexibilidad.

**2. Problemas de Escala** Si $x_t$ tiene rango $[-1, 1]$ pero $\tilde{x}(f)_t$ tiene rango $[-100, 100]$ (por errores numéricos en FFT o parámetros mal normalizados):

$$z_t \approx \tilde{x}(f)_t \quad \text{(la rama con mayor magnitud domina)} \tag{69}$$

Los pesos aprendibles $\alpha, \beta$ permitirían **normalizar** las contribuciones de cada rama.

**3. Degradación del Aprendizaje** Sin pesos aprendibles:

- El gradiente solo fluye a través de las capas anteriores (SSM y extractor espectral)
- Las dos ramas no pueden "aprender a colaborar"
- Si una rama es ruidosa/inútil, no hay forma de reducir su influencia
- Si una rama es insuficiente, no hay forma de aumentar su influencia

**4. Ejemplo Negativo Concreto** Supón que durante el entrenamiento descubrimos:

- Rama temporal: muy informativa para tráfico normal, pero ruidosa para ataques
- Rama frecuencial: débil para normal, pero excelente para ataques

Con pesos aprendibles:

$$z_t[i] = 0{,}8 \cdot x_t[i] + 0{,}2 \cdot \tilde{x}(f)_t[i] \quad \text{(mejor balance)} \tag{70}$$

Con pesos fijos en 1:

$$z_t[i] = x_t[i] + \tilde{x}(f)_t[i] \quad \text{(ambas ramas igual weight, subóptimo)} \tag{71}$$

**Conclusión:**
Permitir que $\alpha, \beta$ sean **aprendibles** es **crucial**. Sin esto:

1. El modelo pierde flexibilidad para balancear ramas
2. Problemas de escala pueden dominar una rama sobre la otra
3. El gradiente no puede ajustar la contribución relativa de cada rama
4. El modelo queda con un balance subóptimo, fijo

Con pesos aprendibles, el modelo **descubre automáticamente** cómo combinar las dos perspectivas (temporal y frecuencial) de manera óptima para el problema en cuestión.

# 6. Arquitectura Completa

## 6.1. Diagrama de Arquitectura

```
                Entrada u_t (batch, D)
                       |
                       v

            Proyección Entrada
            u_proj = Linear(u_t)

                       |

           |                       |
           v                       v
     RAMA TEMPORAL           RAMA FRECUENCIAL

      SSM Select.           FFT
                Top-K
       Block 1            Filtro Compl.
       Block 2            IFFT
       ...
       Block N            Lineal (opt.)

      Output:  x_t          Output:x_f(t)

           |                       |

                       |
                       v

            Fusión Ponderada
            z_t = x_t + x_f
            ,  aprendibles

                       |
                       v

            Cabeza Clasificación
            Linear(z_t) -> logits
            Softmax -> [p_normal,
                        p_anomaly]

                       |
                       v
                Salida: p_t  [0,1]
          (probabilidad de anomalía)
```

## 6.2. Componentes Principales

### 6.2.1. Entrada

Secuencia de características $u_t \in \mathbb{R}^D$ (por ejemplo, 49 características del UNSW-NB15)

### 6.2.2. Proyección de Entrada

Proyección lineal opcional para adaptar dimensionalidad:

$$u_{\text{proj}} = W_{\text{in}} u_t + b_{\text{in}}, \quad W_{\text{in}} \in \mathbb{R}^{D' \times D} \tag{72}$$

(No siempre necesaria si $D$ ya es apropiadad)

### 6.2.3. Rama Temporal

Conjunto de $N$ bloques de SSM selectivo:

- Cada bloque: recurrencia con $\Delta_t, B_t, C_t$ dependientes de entrada
- Activación: SiLU (gated)
- Conexión residual con Layer Normalization
- Salida: $x_t \in \mathbb{R}^D$ (representación temporal)

### 6.2.4. Rama Frecuencial

Extractor de características espectrales:

- FFT de la entrada
- Retener top-$K$ frecuencias
- Aplicar filtro complejo aprendible
- IFFT de vuelta al dominio temporal
- Salida: $\tilde{x}(f)_t \in \mathbb{R}^D$ (representación frecuencial)

### 6.2.5. Fusión

Combinación elemento a elemento con pesos aprendibles:

$$z_t = \alpha \odot x_t + \beta \odot \tilde{x}(f)_t \tag{73}$$

### 6.2.6. Cabeza de Clasificación

Proyección lineal a logits + Softmax:

$$\text{logits} = W_{\text{out}} z_t + b_{\text{out}}, \quad W_{\text{out}} \in \mathbb{R}^{2 \times D} \tag{74}$$

$$p_t = \text{Softmax}(\text{logits}), \quad p_t \in [0,1]^2 \text{ (probabilidades clase)} \tag{75}$$

Se toma $p_t[1]$ como probabilidad de anomalía.

## 6.3. ¿Por qué la rama frecuencial está en paralelo y no en serie?

**Pregunta explícita del documento** *Respuesta:*

### 6.3.1. Configuración en Serie (NO recomendado)

Topología: Entrada → Temporal → Frecuencial → Fusión

$$x_t = \text{Temporal}(u_t) \quad \Rightarrow \quad \tilde{x} = \text{Frecuencial}(x_t) \quad \Rightarrow \quad z_t = \text{Fusión}(\tilde{x}) \tag{76}$$

Problemas:

1. **Pérdida de información:** Aplicar FFT a la salida de la rama temporal destruye las correlaciones temporales finas que la rama temporal extrajo. La transformada de Fourier es una "decisión irreversible" que pierde estructura.
2. **Sesgo temporal a priori:** El modelo procesa primero con lógica temporal, luego intenta extraer periodicidad de esa salida ya procesada. Esto sesga fuertemente hacia la interpretación temporal.
3. **Cuello de botella:** La información debe pasar por un único cuello de botella (la salida de la rama temporal). Cualquier información perdida allí no puede recuperarse en la rama frecuencial.
4. **Falta de perspectivas complementarias:** La rama frecuencial no ve la entrada cruda; ve solo lo que la rama temporal decidió retener.

### 6.3.2. Configuración en Paralelo (RECOMENDADO)

Topología: Entrada → {Temporal, Frecuencial} en paralelo → Fusión

$$u_t \to \begin{cases} x_t = \text{Temporal}(u_t) \\ \tilde{x}(f)_t = \text{Frecuencial}(u_t) \end{cases} \to z_t = \text{Fusión}(x_t, \tilde{x}(f)_t) \tag{77}$$

Ventajas:

1. **Múltiples perspectivas:** Ambas ramas procesan la **entrada cruda** $u_t$, extrayendo información complementaria desde cero.
2. **Sin sesgo:** No hay prejuicio hacia una representación sobre la otra. Ambas tienen acceso idéntico a la entrada.
3. **Mayor simetría:** El modelo trata ambas representaciones como primarias, no como post-procesamiento de la otra.
4. **Capacidad aumentada:** Con $N$ parámetros se extraen dos tipos de representaciones en paralelo; con arquitectura serie, se requeriría pasarlas en serie, reduciendo la expresividad relativa.
5. **Detección dual:**
   - Rama temporal: detecta cambios rápidos, picos anómalos, evolución temporal
   - Rama frecuencial: detecta desviaciones en patrones periódicos, ciclos rotos, comportamientos anómalos que violan periodicidad esperada
   - Fusión: ambas anomalías se combinan para decisión final

**Caso Concreto: Detección de Anomalías en Tráfico**

- **Ataque tipo DDoS (volumétrico):** Causa picos repentinos en volumen de bytes. La rama temporal lo detecta inmediatamente mediante cambios rápidos en estado.
- **Ataque tipo *low-rate*:** Distribuye tráfico anómalo lentamente, manteniendo volumen similar al normal pero *violando la periodicidad esperada* (p. ej., actividad anómala a horas inusuales). La rama temporal podría no detectarlo (los valores parecen normales), pero la rama frecuencial lo capta (la distribución espectral es anómala).
- **Arquitectura paralela:** El modelo puede detectar ambos tipos de ataque independientemente. Con arquitectura serie, el segundo tipo podría no detectarse si la rama temporal no lo ve.

**Conclusión:**
La configuración **paralela** es superior porque permite que ambas ramas (temporal y frecuencial) procesen la entrada cruda de manera independiente, extrayendo perspectivas complementarias sin prejuicios. Esto resulta en un modelo más robusto y expresivo que puede detectar múltiples tipos de anomalías.

# 7. Hiperparámetros

## 7.1. Listado y Clasificación

A continuación presentamos los hiperparámetros del modelo, clasificados por su rol:

| **Hiperparámetro** | **Categoría** | **Valor Inicial** | **Rango** |
|---|---|---|---|
| *Parámetros de Capacidad Modeladora* | | | |
| $N$ (dim. estado) | Capacidad | 64 | [32, 512] |
| $L$ (longitud ventana) | Capacidad | 128 | [64, 512] |
| $K$ (top-K espectral) | Capacidad | 16 | [4, $L/2+1$] |
| $D'$ (dim. proyección entrada) | Capacidad | 32 | [16, 128] |
| *Parámetros de Estabilidad* | | | |
| $T$ (período muestreo) | Estabilidad | 0.01 | [0.001, 0.1] |
| $\lambda_{\text{mín}}$ | Estabilidad | 0.01 | [0.001, 1.0] |
| $\lambda_{\text{máx}}$ | Estabilidad | 1.0 | [0.1, 10.0] |
| *Parámetros de Entrenamiento* | | | |
| lr (learning rate) | Entrenamiento | $10^{-3}$ | [$10^{-4}$, $10^{-2}$] |
| $\text{clip}_{\text{grad}}$ | Entrenamiento | 1.0 | [0.5, 5.0] |
| batch_size | Entrenamiento | 32 | [16, 128] |
| *Parámetros de Regularización* | | | |
| $p_{\text{dropout}}$ | Regularización | 0.1 | [0.0, 0.5] |
| $\lambda_{\text{L2}}$ (weight decay) | Regularización | $10^{-4}$ | [$10^{-6}$, $10^{-3}$] |

Cuadro 1: Hiperparámetros del modelo de detección de anomalías.

## 7.2. Justificación de Valores Iniciales

### 7.2.1. Parámetros de Capacidad

$N = 64$ **(dimensión del estado latente)**

- El dataset UNSW-NB15 tiene 49 características de entrada
- Una dimensión latente de 64 proporciona espacio suficiente para comprimir y transformar la entrada sin perder información crítica
- Es un tamaño estándar en modelos de espacio de estado modernos (comparable a configuraciones en mamba-ssm, S4)
- No es tan grande como para causar overfitting o costo computacional excesivo ($\sim 64^2 = 4096$ operaciones por paso)
- Rango [32, 512] permite experimentación: 32 es comprimido (menos capacidad), 512 es expresivo (más parámetros)

$L = 128$ **(longitud de ventana temporal)**

- En tráfico de red, las anomalías pueden desarrollarse en escalas de segundos a minutos

- A 100 Hz de muestreo (típico en análisis de flujos), 128 muestras $\approx$ 1.28 segundos de historia
- Captura dinámicas de corto-plazo (cambios rápidos en volumen, puertos, etc.) sin ser tan largo como para incluir demasiada historia irrelevante
- Rango [64, 512]: 64 es muy corto (puede perder dependencias importantes), 512 es muy largo (costo computacional alto)

$K = 16$ **(top-$K$ frecuencias retenidas)**

- Con $L = 128$, hay $L/2 + 1 = 65$ coeficientes de frecuencia únicos
- Retener 16 de 65 es una compresión de $\sim 75\,\%$: agresiva pero razonable para eliminar ruido
- Captura las periodicidades más prominentes (ciclos diarios, horarios, patrones de sincronización) sin ruido de alta frecuencia
- Rango [4, 65]: 4 es muy comprimido (pierde mucha información), 65 es sin compresión (sin beneficio frecuencial)

$D' = 32$ **(dimensión de proyección de entrada)**

- Proyecta las 49 características de entrada a 32 dimensiones
- Ligera compresión, pero el modelo tiene suficientes parámetros para capturar toda la información importante
- Reduce dimensionalidad para estabilidad numérica en cálculos posteriores

### 7.2.2. Parámetros de Estabilidad

$T = 0{,}01$ **(período de muestreo discreta)**

- Determina la escala de discretización ZOH: $A_d = e^{-T\mathrm{diag}(\lambda)}$
- Con $T = 0{,}01$ y $\lambda_i \in [0{,}01, 1{,}0]$, los eigenvalores discretos son:

$$\mu_i = e^{-\lambda_i T} \in [e^{-0{,}01}, e^{-0{,}0001}] \approx [0{,}99, 0{,}9999] \tag{78}$$

- Este rango asegura estabilidad (todos $< 1$) y buena retención de información (valores cercanos a 1)
- Si $T$ fuera demasiado grande (e.g., $T = 1$), los eigenvalores serían $\mu_i \approx [0{,}37, 0{,}9999]$, más variable
- Si $T$ fuera demasiado pequeño (e.g., $T = 0{,}0001$), los eigenvalues serían $\approx 1$, casi sin decaimiento

$\lambda_{\text{mín}} = 0{,}01$, $\lambda_{\text{máx}} = 1{,}0$

- Define el rango de escalas temporales que el modelo puede capturar
- Con $\lambda_{\text{mín}} = 0{,}01$: dependencias largas ($\tau \sim 1/\lambda_{\text{mín}} = 100$ pasos de tiempo)
- Con $\lambda_{\text{máx}} = 1{,}0$: dependencias cortas ($\tau \sim 1/\lambda_{\text{máx}} = 1$ paso)
- Un factor de $\approx 100$ entre mínimo y máximo permite capturar dinámicas en múltiples escalas
- La inicialización de $\lambda_i$ se hace muestreando uniformemente en log-space: $\lambda_i \sim \mathcal{U}(\log \lambda_{\text{mín}}, \log \lambda_{\text{máx}})$

### 7.2.3. Parámetros de Entrenamiento

**lr** $= 10^{-3}$ **(learning rate de Adam)**

- Valor estándar recomendado por la literatura de aprendizaje profundo
- Suficientemente pequeño para garantizar convergencia suave sin divergencia
- Suficientemente grande para que el entrenamiento sea razonablemente rápido
- Rango $[10^{-4}, 10^{-2}]$: valores más pequeños convergen lentamente; valores más grandes pueden causar overshoot y divergencia

**clip$_{\text{grad}}$** $= 1{,}0$ **(recorte de norma de gradiente)**

- Limita la norma del vector de gradiente a máximo 1.0 antes de actualización
- Previene explosión de gradientes, especialmente importante en recurrencias largas
- En nuestro SSM, este riesgo es menor que en RNNs tradicionales, pero aún útil como precaución
- Si el gradiente $\|\nabla L\| > 1$, se normaliza: $\nabla L \leftarrow \nabla L / \|\nabla L\|$

**batch_size** $= 32$

- Balance entre velocidad de entrenamiento y estabilidad numérica
- 32 es suficiente para buen estimado de gradiente (no es demasiado pequeño como 1, ni demasiado grande como 512)
- Rango [16, 128]: tamaños menores dan gradientes más ruidosos; mayores requieren más memoria GPU

### 7.2.4. Parámetros de Regularización

$p_{\mathbf{dropout}} = 0{,}1$ **(probabilidad de dropout)**

- Desconecta aleatoriamente 10 % de activaciones durante entrenamiento
- Previene co-adaptación excesiva de neuronas (overfitting)
- Leve (10 %), para no inhibir el aprendizaje demasiado
- Rango [0.0, 0.5]: sin dropout (0 %) permite overfitting; dropout alto ($> 0{,}5$) puede dañar la capacidad modeladora

$\lambda_{\mathbf{L2}} = 10^{-4}$ **(weight decay)**

- Regularización que penaliza grandes pesos: $L_{\text{total}} = L_{\text{clasif}} + \lambda_{\text{L2}}\|\theta\|_2^2$
- Ayuda a mantener pesos pequeños y generalizables.
- Muy pequeño ($10^{-4}$) para no penalizar excesivamente pesos grandes necesarios en SSM.
- Rango $[10^{-6}, 10^{-3}]$: valores menores son despreciables; valores mayores pueden comprometer el aprendizaje.

## 7.3. Inicialización de Parámetros Clave

### 7.3.1. Eigenvalues $\lambda_i$

Inicializar uniformemente en log-space:

$$\lambda_i \sim \mathcal{U}(\log \lambda_{\text{mín}}, \log \lambda_{\text{máx}}) = \mathcal{U}(\log 0{,}01, \log 1{,}0) = \mathcal{U}(-4{,}6, 0) \tag{79}$$

Luego exponenciar: $\lambda_i = \exp(\text{sample})$ para obtener $\lambda_i \in [0{,}01, 1{,}0]$.

Justificación: log-uniform evita aglomeración de valores cercanos a 1.0 o 0.01, proporcionando cobertura uniforme en las escalas temporales.

### 7.3.2. Pesos de Fusión $\alpha, \beta$

Inicializar con:

$$\alpha_i \sim \mathcal{U}(0{,}3, 0{,}7), \quad \beta_i \sim \mathcal{U}(0{,}3, 0{,}7) \tag{80}$$

O, más simplemente: $\alpha_i = 0{,}5, \beta_i = 0{,}5$ para todo $i$.

Justificación: balance inicial entre ramas. Los valores se ajustarán durante entrenamiento según gradiente.

**Resumen de Inicialización:**
Los hiperparámetros iniciales reflejan un balance cuidadoso entre **capacidad modeladora** (suficientes parámetros para aprender), **estabilidad numérica** (eigenvalues que garantizan estabilidad), y **regularización** (dropout, weight decay para evitar overfitting).
Estos valores son puntos de partida razonables. El tuning fino mediante validación cruzada o búsqueda de hiperparámetros (grid search, random search, Bayesian optimization) puede mejorar el desempeño.

# 8. Filtro de Kalman como Post-Procesamiento

## 8.1. Motivación

El modelo entrenado produce, para cada ventana temporal, una **probabilidad** $p_t \in [0, 1]$ de que esa ventana sea anómala. En la práctica, estas probabilidades son **ruidosas**: oscilan significativamente de ventana a ventana, incluso cuando el **verdadero estado subyacente del sistema cambia lentamente**.

**Ejemplo:** Supongamos que entre los tiempos $t-5$ y $t+5$ ocurre un ataque gradual que aumenta lentamente. Las probabilidades predichas podrían ser:

$$p_{t-5} = 0{,}3, \quad p_{t-4} = 0{,}4, \quad p_{t-3} = 0{,}2, \quad p_{t-2} = 0{,}5, \quad p_{t-1} = 0{,}35, \quad p_t = 0{,}6 \tag{81}$$

Estas oscilaciones ($0.3 \to 0.4 \to 0.2 \to 0.5$, etc.) reflejan **ruido de predicción**, no cambios reales en el estado del sistema.

**Solución:** Aplicar un **filtro de Kalman** que modele la *verdadera* probabilidad de anomalía como una variable latente que evoluciona lentamente, y que sea observada con ruido a través de las predicciones del modelo.

## 8.2. Modelo Estocástico Escalar

Formulamos un modelo de espacio de estados escalar para la **verdadera** probabilidad latente $s_t \in \mathbb{R}$ (no necesariamente en $[0, 1]$ para la matemática; la restricción se aplicará al final):

### 8.2.1. Ecuación de Estado (Dinámica del Sistema)

$$s_{t+1} = s_t + w_t, \quad w_t \sim \mathcal{N}(0, \sigma_w^2) \tag{82}$$

Interpretación:

- La verdadera probabilidad de anomalía $s_t$ sigue una **caminata aleatoria gaussiana** (random walk)
- En cada paso, puede cambiar por una cantidad pequeña $w_t$ distribuida gaussianamente
- La varianza $\sigma_w^2$ controla cuán rápido puede cambiar el verdadero estado
- Si $\sigma_w^2$ es pequeña, $s_t$ cambia lentamente (sistema estable)
- Si $\sigma_w^2$ es grande, $s_t$ puede cambiar rápidamente (sistema volátil)

### 8.2.2. Ecuación de Observación (Medición Ruidosa)

$$p_t = s_t + v_t, \quad v_t \sim \mathcal{N}(0, \sigma_v^2) \tag{83}$$

Interpretación:

- Las predicciones del modelo $p_t$ son observaciones ruidosas del verdadero estado $s_t$
- El ruido de observación $v_t$ es gaussiano con varianza $\sigma_v^2$
- Si $\sigma_v^2$ es pequeña, el modelo es confiable (poco ruido)
- Si $\sigma_v^2$ es grande, el modelo es ruidoso (mucho desacuerdo con el verdadero estado)

### 8.2.3. Supuestos

- $w_t$ y $w_s$ son independientes $\forall s \neq t$ (ruido de proceso temporalmente independiente)
- $v_t$ y $v_s$ son independientes $\forall s \neq t$ (ruido de medición temporalmente independiente)
- $w_t$ y $v_s$ son independientes $\forall s, t$ (ruido de proceso y medición independientes entre sí)
- $w_0 \sim \mathcal{N}(0, \sigma_w^2)$, inicialización gaussiana

## 8.3. Filtro de Kalman: Derivación Completa

El filtro de Kalman estima óptimamente (en el sentido de mínimo error cuadrático medio) el estado latente $s_t$ a partir de observaciones ruidosas $p_0, p_1, \ldots, p_t$.

### 8.3.1. Notación

- $\hat{s}_{t|t}$: **estimación a posteriori** del estado en tiempo $t$ (después de ver observación $p_t$)
- $\hat{s}_{t|t-1}$: **estimación a priori** (predicción) del estado en tiempo $t$ (antes de ver observación $p_t$)
- $P_{t|t}$: **varianza del error a posteriori** (incertidumbre después de ver la observación)
- $P_{t|t-1}$: **varianza del error a priori** (incertidumbre de predicción)
- $K_t$: **ganancia de Kalman** en tiempo $t$ (cuánto confiar en la nueva observación)

### 8.3.2. Paso 1: Predicción (A Priori)

Dada la estimación anterior $\hat{s}_{t-1|t-1}$, predecimos el estado en tiempo $t$:

**Ecuación de predicción:** Aplicamos la dinámica del sistema sin conocer $w_{t-1}$:

$$\hat{s}_{t|t-1} = \hat{s}_{t-1|t-1} + \mathbb{E}[w_{t-1}] \tag{84}$$

Dado que $w_{t-1} \sim \mathcal{N}(0, \sigma_w^2)$, tenemos $\mathbb{E}[w_{t-1}] = 0$:

$$\boxed{\hat{s}_{t|t-1} = \hat{s}_{t-1|t-1}} \tag{85}$$

Interpretación: sin nueva información, la mejor predicción del estado futuro es el estado actual.

**Varianza de predicción:** Tomamos varianza en ambos lados de $s_t = \hat{s}_{t-1|t-1} + w_{t-1}$:

$$\begin{aligned}
P_{t|t-1} &= \mathrm{Var}(s_t|\mathrm{obs}_{t-1}) && (86)\\
&= \mathrm{Var}(\hat{s}_{t-1|t-1} + w_{t-1}|\mathrm{obs}_{t-1}) && (87)\\
&= \mathrm{Var}(\hat{s}_{t-1|t-1}|\mathrm{obs}_{t-1}) + \mathrm{Var}(w_{t-1}) \quad \text{(independencia)} && (88)\\
&= P_{t-1|t-1} + \sigma_w^2 && (89)
\end{aligned}$$

$$\boxed{P_{t|t-1} = P_{t-1|t-1} + \sigma_w^2} \tag{90}$$

Interpretación: la incertidumbre crece en cantidad $\sigma_w^2$ porque el sistema es volátil. Si no vemos nuevas observaciones, nuestra confianza en el estado disminuye.

### 8.3.3. Paso 2: Ganancia de Kalman (Óptima)

La actualización debe ponderar entre confiar en la predicción a priori $\hat{s}_{t|t-1}$ y en la nueva observación $p_t$:

$$\hat{s}_{t|t} = \hat{s}_{t|t-1} + K_t(p_t - \hat{s}_{t|t-1}) \tag{91}$$

donde $K_t \in [0, 1]$ es la **ganancia de Kalman**:

- Si $K_t = 0$: no confiar en $p_t$; mantener predicción $\hat{s}_{t|t-1}$
- Si $K_t = 1$: confiar completamente en $p_t$; ignorar predicción
- Si $K_t \in (0, 1)$: compromiso

**Derivación de $K_t$ Óptima:** El error a posteriori es:

$$\begin{aligned}
e_{t|t} &= s_t - \hat{s}_{t|t} && (92)\\
&= s_t - \hat{s}_{t|t-1} - K_t(p_t - \hat{s}_{t|t-1}) && (93)\\
&= (s_t - \hat{s}_{t|t-1}) - K_t(s_t + v_t - \hat{s}_{t|t-1}) \quad \text{(sustituir } p_t = s_t + v_t\text{)} && (94)\\
&= (1 - K_t)(s_t - \hat{s}_{t|t-1}) - K_t v_t && (95)\\
&= (1 - K_t)\epsilon_{t|t-1} - K_t v_t && (96)
\end{aligned}$$

donde $\epsilon_{t|t-1} = s_t - \hat{s}_{t|t-1}$ es el error de predicción, con varianza $P_{t|t-1}$.

La varianza del error a posteriori es:

$$P_{t|t} = \mathbb{E}[e_{t|t}^2] \tag{97}$$
$$= (1 - K_t)^2 P_{t|t-1} + K_t^2 \sigma_v^2 \quad \text{(independencia de } \epsilon_{t|t-1} \text{ y } v_t) \tag{98}$$

Minimizamos respecto a $K_t$:

$$\frac{\partial P_{t|t}}{\partial K_t} = -2(1 - K_t)P_{t|t-1} + 2K_t\sigma_v^2 = 0 \tag{99}$$

Resolvemos para $K_t$:

$$-2(1 - K_t)P_{t|t-1} + 2K_t\sigma_v^2 = 0 \tag{100}$$
$$2K_t(P_{t|t-1} + \sigma_v^2) = 2P_{t|t-1} \tag{101}$$
$$K_t = \frac{P_{t|t-1}}{P_{t|t-1} + \sigma_v^2} \tag{102}$$

$$\boxed{K_t = \frac{P_{t|t-1}}{P_{t|t-1} + \sigma_v^2}} \tag{103}$$

Interpretación:

- Si $P_{t|t-1}$ es grande (mucha incertidumbre en predicción), $K_t \to 1$ (confiar en observación)
- Si $P_{t|t-1}$ es pequeña (poca incertidumbre en predicción), $K_t \to 0$ (confiar en predicción)
- Si $\sigma_v^2$ es grande (mucho ruido de medición), $K_t \to 0$ (no confiar en observación ruidosa)
- Si $\sigma_v^2$ es pequeña (poco ruido), $K_t \to$ mayor (confiar más en observación confiable)

### 8.3.4. Paso 3: Actualización (A Posteriori)

Usando la ganancia óptima $K_t$:

$$\boxed{\hat{s}_{t|t} = \hat{s}_{t|t-1} + K_t(p_t - \hat{s}_{t|t-1})} \tag{104}$$

Para la varianza a posteriori, sustituimos $K_t$ óptimo:

$$P_{t|t} = (1 - K_t)^2 P_{t|t-1} + K_t^2 \sigma_v^2 \tag{105}$$
$$= \left(1 - \frac{P_{t|t-1}}{P_{t|t-1} + \sigma_v^2}\right)^2 P_{t|t-1} + \left(\frac{P_{t|t-1}}{P_{t|t-1} + \sigma_v^2}\right)^2 \sigma_v^2 \tag{106}$$

Simplificando (álgebra omitida):

$$P_{t|t} = \frac{P_{t|t-1}\sigma_v^2}{P_{t|t-1} + \sigma_v^2} \tag{107}$$

O alternativamente:

$$\boxed{P_{t|t} = (1 - K_t)P_{t|t-1}} \tag{108}$$

Interpretación: la varianza a posteriori siempre es menor que la a priori porque hemos visto una nueva observación y reducimos la incertidumbre.

## 8.4. Resumen del Algoritmo

**Filtro de Kalman Escalar para Suavizado de Probabilidades**
**Inicialización:**

$$\hat{s}_{0|-1} = 0{,}5 \quad \text{(asumimos probabilidad inicial 50 \%)} \tag{109}$$
$$P_{0|-1} = 1{,}0 \quad \text{(varianza inicial grande, mucha incertidumbre)} \tag{110}$$

**Para cada tiempo** $t = 0, 1, 2, \ldots, T$:
**1. Predicción (a priori):**

$$\hat{s}_{t|t-1} = \hat{s}_{t-1|t-1} \tag{111}$$
$$P_{t|t-1} = P_{t-1|t-1} + \sigma_w^2 \tag{112}$$

**2. Ganancia de Kalman:**

$$K_t = \frac{P_{t|t-1}}{P_{t|t-1} + \sigma_v^2} \tag{113}$$

**3. Actualización (a posteriori):**

$$\hat{s}_{t|t} = \hat{s}_{t|t-1} + K_t(p_t - \hat{s}_{t|t-1}) \tag{114}$$
$$P_{t|t} = (1 - K_t)P_{t|t-1} \tag{115}$$

**Salida:** $\hat{s}_{0|0}, \hat{s}_{1|1}, \ldots, \hat{s}_{T|T}$ (secuencia suavizada de probabilidades latentes)
**Para clasificación binaria:** $y_t = \mathbb{1}[\hat{s}_{t|t} > 0{,}5]$ (umbral en 0.5)

## 8.5. Análisis de Casos Extremos

### 8.5.1. Pregunta 1: ¿Qué pasa cuando $\sigma_w^2 \to 0$?

Cuando el ruido de proceso tiende a cero, el estado latente evoluciona como una constante pura.

**Comportamiento de la ganancia:** La varianza de predicción es:

$$P_{t|t-1} = P_{t-1|t-1} + \sigma_w^2 \to P_{t-1|t-1} \quad (\text{cuando } \sigma_w^2 \to 0) \tag{116}$$

Si comenzamos con $P_{0|-1} = 1{,}0$ pequeño (relativo), entonces:

$$P_{t|t-1} \to P^* \quad \text{(converge a constante)} \tag{117}$$

La ganancia de Kalman tiende a:

$$K_t = \frac{P_{t|t-1}}{P_{t|t-1} + \sigma_v^2} \to \frac{P^*}{P^* + \sigma_v^2} \equiv K^* \tag{118}$$

Como $P^*$ se mantiene finito y $\sigma_v^2$ es fijo, $K^* < 1$ pero no necesariamente pequeño.

**Efecto en la estimación:** La actualización es:

$$\hat{s}_{t|t} = \hat{s}_{t|t-1} + K^*(p_t - \hat{s}_{t|t-1}) = (1 - K^*)\hat{s}_{t|t-1} + K^* p_t \tag{119}$$

Es un **promedio exponencial ponderado** entre la predicción anterior y la observación actual:

$$\hat{s}_{t|t} \approx \mathrm{EMA}(p_t, \alpha = K^*) \tag{120}$$

Con $K^*$ pequeño (mucho más pequeño que 1), el filtro **desconfía de las nuevas observaciones** y evoluciona lentamente.

**Respuesta:** *Cuando $\sigma_w^2 \to 0$:*

- La ganancia de Kalman $K_t \to K^* < 1$ (fija)
- El filtro usa un promedio exponencial con peso pequeño en observaciones nuevas
- **Comportamiento: suavizado máximo, sigue lentamente (o no sigue) cambios en observaciones**
- El filtro asume que el verdadero estado es prácticamente constante y que las observaciones ruidosas oscilan alrededor de ese valor fijo

### 8.5.2. Pregunta 2: ¿Qué pasa cuando $\sigma_w^2 \to \infty$?

Cuando el ruido de proceso tiende a infinito, el estado latente puede cambiar de forma radicalmente impredecible.

**Comportamiento de la ganancia:** La varianza de predicción es:

$$P_{t|t-1} = P_{t-1|t-1} + \sigma_w^2 \to \infty \quad (\text{cuando } \sigma_w^2 \to \infty) \tag{121}$$

Incluso después de actualización, la varianza a posteriori es:

$$P_{t|t} = (1 - K_t)P_{t|t-1} \tag{122}$$

con $K_t = \frac{P_{t|t-1}}{P_{t|t-1}+\sigma_v^2} \to \frac{\infty}{\infty+\text{const}} = 1$.

**Efecto en la estimación:** La actualización es:

$$\hat{s}_{t|t} = \hat{s}_{t|t-1} + K_t(p_t - \hat{s}_{t|t-1}) \to \hat{s}_{t|t-1} + 1 \cdot (p_t - \hat{s}_{t|t-1}) = p_t \tag{123}$$

El filtro simplemente devuelve la observación cruda sin ningún suavizado.

**Respuesta:** *Cuando $\sigma_w^2 \to \infty$:*

- La ganancia de Kalman $K_t \to 1$
- El filtro confía completamente en las observaciones nuevas
- **Comportamiento: suavizado mínimo, sigue inmediatamente cualquier cambio (incluso ruido)**
- El filtro asume que el verdadero estado puede cambiar arbitrariamente y por eso da peso máximo a las observaciones recientes

### 8.5.3. Pregunta 3: ¿Qué relación $\sigma_w^2/\sigma_v^2$ favorece cada régimen?

El balance entre $\sigma_w^2$ (volatilidad del sistema) y $\sigma_v^2$ (ruido de medición) determina el comportamiento:

**Ratio Pequeño: $\sigma_w^2/\sigma_v^2 \ll 1$ (sistema estable, ruido de medición alto)**

- El verdadero estado cambia lentamente ($\sigma_w^2$ pequeño)
- Las mediciones son ruidosas ($\sigma_v^2$ grande)
- El filtro tiende a suavizar agresivamente, ignorando cambios rápidos en $p_t$ como ruido

**Ratio Grande: $\sigma_w^2/\sigma_v^2 \gg 1$ (sistema volátil, ruido de medición bajo)**

- El verdadero estado cambia rápidamente ($\sigma_w^2$ grande)
- Las mediciones son confiables ($\sigma_v^2$ pequeño)
- El filtro tiende a seguir rápidamente las observaciones, creyendo que cambios son reales, no ruido

**Ratio Intermedio: $\sigma_w^2/\sigma_v^2 \approx 1$**

- El filtro balancea ambas fuentes de incertidumbre de manera equilibrada

### 8.5.4. Pregunta 4: ¿En qué escenarios del problema real esperarían que cada régimen sea preferible?

**Suavizado Agresivo ($\sigma_w^2$ pequeño): Escenario 1: Tráfico de red normal con anomalías puntuales**

- El comportamiento base es predecible y periódico (ciclos diarios, patrones horarios)
- Las anomalías son eventos breves, localizados (un ataque que dura segundos o minutos)
- Si aplicamos suavizado agresivo, el filtro no sobrerreacciona a cada pico, sino que requiere **cambios sostenidos** para cambiar su estimación
- Ventaja: reduce falsos positivos (evita alarmar por ruido)
- Desventaja: puede retardar la detección de anomalías que se desarrollan lentamente

**Seguimiento Rápido ($\sigma_w^2$ grande): Escenario 2: Tráfico altamente dinámico con ataques sostenidos**

- El tráfico legítimo es altamente variable (redes con picos esporádicos normales)
- Los ataques de interés son sostenidos y evolucionantes (DDoS gradual, exfiltración lenta)

- Si aplicamos seguimiento rápido, el filtro adapta rápidamente su estimación, permitiendo detectar cambios prolongados
- Ventaja: detecta ataques que se desarrollan gradualmente sin demora
- Desventaja: puede generar más falsos positivos en tráfico ruidoso

**Balance Intermedio: Escenario 3: Tráfico con cambios graduales y anomalías sostenidas**

- El tráfico normal cambia a lo largo del día de manera gradual (rampa de usuarios mañana, caída nocturna)
- Las anomalías de interés son sostenidas (botnets, conexiones persistentes anómalas)
- Un balance intermedio permite seguir cambios graduales sin sobrerreaccionar a fluctuaciones de corto plazo

**Conclusión:**
La elección de $(\sigma_w^2, \sigma_v^2)$ debe reflejar las **características esperadas del problema real**:

- Si las anomalías son **puntuales y el sistema estable**: elegir $\sigma_w^2$ pequeño
- Si las anomalías son **sostenidas y el sistema volátil**: elegir $\sigma_w^2$ grande
- Explorar múltiples valores en experimentos (bloque 3) para encontrar el balance óptimo para tu dataset

# Conclusión

Este documento ha presentado la derivación matemática completa del Bloque 1 del proyecto, cubriendo:

1. Modelado en espacio de estado discreto con garantías de estabilidad
2. Parametrización selectiva dependiente de entrada para adaptabilidad
3. Análisis espectral mediante FFT/DFT y extracción de características periódicas
4. Fusión ponderada con parámetros aprendibles
5. Arquitectura completa con dos ramas en paralelo
6. Hiperparámetros del modelo con justificaciones de valores iniciales
7. Filtro de Kalman escalar para post-procesamiento y suavizado de predicciones
8. Análisis de casos extremos del filtro de Kalman

Todos los requisitos del sistema (R1-R5) se satisfacen:

- **R1**: SSM con complejidad $\mathcal{O}(L \cdot N^2)$ lineal en $L$
- **R2**: Estabilidad garantizada por parametrización diagonal
- **R3**: Extracción explícita de componentes periódicos en dominio frecuencial
- **R4**: Fusión adaptativa con coeficientes $\alpha, \beta$ aprendibles
- **R5**: Filtro de Kalman como post-procesamiento recursivo óptimo

Este bloque proporciona la base teórica para la implementación en PyTorch (Bloque 2) y la evaluación experimental (Bloque 3).